



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» Кафедра 806
Группа М8О-411Б-20 Направление подготовки 02.03.02 Фундаментальная информатика
и информационные технологии
Профиль Информатика и компьютерные науки
Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

на тему: «Детектирование и трекинг движущихся объектов методами
компьютерного зрения»

Автор ВКРБ: Нурмухамедов Абубакир Бобурович (_____)
Руководитель: Филимонов Александр Борисович (_____)

К защите допустить

Заведующий кафедрой № 806 «Вычислительная математика
и программирование» Крылов Сергей Сергеевич (_____)

XX мая 2024 года

Москва 2024



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» Кафедра 806
Группа М8О-411Б-20 Направление подготовки 02.03.02 Фундаментальная информатика
и информационные технологии

Профиль Информатика и компьютерные науки

Квалификация бакалавр

УТВЕРЖДАЮ

Заведующий кафедрой № 806 «Вычислительная
математика и программирование» (_____) С.С. Крылов

14 октября 2023 года

ЗАДАНИЕ

на выпускную квалификационную работу бакалавра

Обучающийся Нурмухамедов Абубакир Бобурович

Руководитель Филимонов Александр Борисович, д.т.н., с.н.с., профессор, каф. 806

1. Наименование темы: «Детектирование и трекинг движущихся объектов методами
компьютерного зрения».

2. Срок сдачи обучающимся законченной работы: 24 апреля 2024 года

3. Задание и исходные данные к работе

Положение о порядке проведения итоговой аттестации по основным образовательным
программам высшего образования – программам бакалавриата, специалитета и
магистратуры ФГБОУ ВО «Московский авиационный институт» (01.09.2021).

Данные и источники данных:

- федеральные авиационные правила «Требования авиационной безопасности к аэропортам»;
- характеристики IP-видеокамер высокого разрешения;
- техническая документация по библиотеке OpenCV;
- публикации по теме ВКР.

Условия и ограничения на разработку:

- оперативный контроль и фиксирование обстановки в зоне видеонаблюдения с целью
предотвращения проникновения на территорию (охрана периметра).

Функциональные требования к системе (алгоритмам) видеонаблюдения:

- функции видеонаблюдения системы охраны территории аэропорта;
- большая протяженность охраняемой зоны;
- обработка видеоданных в реальном масштабе времени;
- разработка программного обеспечения на языке Python.

Перечень иллюстративно-графических материалов

№ п/п	Наименование	Количество листов (слайдов)
1	Постановка задачи видеонаблюдения в охраняемых зонах аэропорта	1

№ п/п	Наименование	Количество листов (слайдов)
2	Архитектура системы видеонаблюдения.	1
3	Метод детекции движущихся объектов	1
4	Архитектура нейронной сети YOLO	1
5	Метод формирования датасета визуальных образов	1
6	Распознавание объектов в зоне видеоконтроля	1
7	Видеосопровождение человека в зоне видеоконтроля	1
8	Результаты вычислительных экспериментов	1

4. Перечень подлежащих разработке разделов и этапы выполнения работы:

№ п/п	Наименование раздела или этапа	Трудоемкость в % от полной трудо- емкости квалифи- кационной работы	Срок выполнения	Примечание
1	Анализ современного состояния исследуемой проблемы, постановка задач, подлежащих разработки	15	18.12.2023	
2	Разработка алгоритмов решения поставленных задач и их программная реализация	35	13.02.2024	
3	Результаты решения поставленных задач и их оценка	25	27.03.2024	
4	Введение (автореферат работы)	15	10.04.2024	
5	Заключение (краткие выводы и перспективы дальнейшей разработки темы)	10	17.04.2024	

5. Исходные материалы и пособия:

1. Солем Я.Э. Программирование компьютерного зрения на языке Python / пер. с англ. Слинкин А.А. – М.: ДМК Пресс, 2016. – 312с.
2. YOLO: Real-Time Object Detection.URL: <https://pjreddie.com/darknet/yolo/>.
- 3 Yilmaz A., Javed O., Shah M. Object tracking: A survey // ACM Comput. Surv. 2006. V. 38. N. 4. – P. 1 - 45.
4. Ramsey Faragher Understanding the Basis of the Kalman Filter Via a Simple and Intuitive Derivation // IEEE SIGNAL PROCESSING MAGAZINE. 2012. – P. 128-132.
5. Садыхов Р.Х., Кучук С.А. Системы видеонаблюдения. Состояние, проблемы и технические средства обработки изображений//Информатика. 2013. № 3. – С. 35.
6. Massimo Piccardi Background subtraction techniques: a review // 2004 IEEE International Conference on Systems, Man and Cybernetics. 2004. – P. 3099-3104.
7. Власов С.О., Гладышев А.И., Богуславский А.А., Соколов С.М. Решение задачи обнаружения объекта с помощью нейросетевых технологий // Препринты ИПМ им. М.В.Келдыша 2023. № 16. С. 1-27.

6. Дата выдачи задания: 14 октября 2023 года

Руководитель (_____) А. Б. Филимонов

Задание принял к исполнению (_____) А.Б.Нурмухамедов

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 44 страниц, 32 рисунков, 0 таблиц, 22 использованных источников, 1 приложения.

АЛГОРИТМ ДЕТЕКТИРОВАНИЯ, АЛГОРИТМ РАСПОЗНАВАНИЯ, АЛГОРИТМ ТРЕКИНГА, МАШИНОЕ ОБУЧЕНИЕ.

Объектом исследования в данной работе является система обеспечения безопасности в аэропортах.

Цель работы – реализация алгоритмов для системы видеонаблюдения охраняемых зон аэропорта.

Для достижения поставленной цели были проведены исследования в области компьютерного зрения, обработки изображений и видео, а также применены современные методы машинного обучения и нейронных сетей. Основное содержание работы состояло в разработке алгоритмов детектирования, распознавания и трекинга.

Основными результатами работы, полученными в процессе разработки, являются реализация системы детектирования движущихся объектов в зоне видеонаблюдения, создание модуля распознавания объектов, который обеспечивает классификацию обнаруженных объектов на основе заранее обученной нейронной сети, внедрение алгоритма трекинга объектов, позволяющего отслеживать движение объектов в реальном времени с использованием фильтра Калмана, подготовка и аннотирование специализированного датасета изображений и видео для обучения и тестирования нейронной сети YOLO.

Данные результаты разработки предназначены для улучшения систем видеонаблюдения в охраняемых зонах аэропорта.

Применение результатов данной работы позволяет повысить эффективность систем видеонаблюдения охраняемых зон, обеспечивая быстрое и точное обнаружение, распознавание и отслеживание движущихся объектов.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	7
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	8
ВВЕДЕНИЕ	2
1 Система охранного видеонаблюдения аэропорта	4
1.1 Проблемы видеонаблюдения в аэропортах	4
1.2 Архитектура системы охранного видеонаблюдения аэропорта.....	6
1.3 Решаемые задачи обработки видеоданных.....	8
2 Обнаружение проникновения в охраняемую зону.....	10
2.1 Основы детектирования объектов	10
2.2 Алгоритм детектирования	11
2.2.1 Вычитание фона	11
2.2.2 Обнаружение контуров	13
2.2.3 Рисования контуров	14
2.2.4 Улучшение обнаружения контуров.....	15
3 Распознавание движущихся объектов.....	20
3.1 Выбор метода распознавания	20
3.2 Обзор нейронной сети YOLO.....	21
3.3 Обучение нейронной сети и используемый датасет	23
3.4 Подключение обученной модели	25
3.4 Результаты.....	26
4 Трекинг объектов наблюдения	28
4.1 Фильтр Калмана	28
4.2 Реализация алгоритма трекинга.....	30
4.3 Сохранение записей обнаружений в базу данных	33
4.4 Уведомление операторам об обнаружении	35
4.5 Результаты.....	36

ЗАКЛЮЧЕНИЕ.....	40
СПИСОК ЛИТЕРАТУРЫ.....	41
ПРИЛОЖЕНИЕ	43

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяются следующие термины с соответствующими определениями:

Датасет	– набор данных, используемых для обучения и тестирования моделей машинного обучения или компьютерного зрения
Нейронная сеть	– алгоритм машинного обучения, моделирующий работу нервной системы человека, используемый для решения различных задач, включая классификацию, детектирование и распознавание объектов

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяются следующие сокращения и обозначения:

OpenCV	– Open Source Computer Vision Library
YOLO	– You Only Live Once

ВВЕДЕНИЕ

В современном мире безопасность является одним из приоритетов, особенно в таких местах, как аэропорты, где обеспечение безопасности оборудования и людей играет решающую роль. Одним из наиболее эффективных методов обеспечения безопасности является использование технологий компьютерного зрения для детектирования и трекинга движущихся объектов.

Системы детектирования и трекинга движущихся объектов с использованием компьютерного зрения становятся важным инструментом для обеспечения безопасности в различных сферах, включая транспортные узлы, такие как аэропорты. Актуальность данной темы обусловлена несколькими факторами:

Во-первых, угрозы безопасности в современном мире продолжают расти. Террористические акты, акты вандализма, кражи и другие непредвиденные ситуации становятся все более частыми. Эффективные системы детектирования и трекинга могут помочь в раннем обнаружении и предотвращении таких угроз.

Во-вторых, в аэропортах находится ценное оборудование и техника, такие как самолеты, технические средства обслуживания и различные материалы. Обеспечение их безопасности имеет критическое значение для непрерывной работы аэропортов и предотвращения экономических потерь.

В-третьих, законодательство в области безопасности авиации строго регулирует требования к безопасности в аэропортах. Разработка и внедрение передовых систем безопасности помогают администрации аэропортов соответствовать данным требованиям.

Внедрение систем детектирования и трекинга движущихся объектов в аэропортах - это актуальная задача, которая может значительно улучшить уровень безопасности и эффективности деятельности воздушных транспортных узлов. Данная работа посвящена проблематике проектирования

системы видеонаблюдения для обеспечения безопасности в аэропорту с использованием библиотеки OpenCV и архитектуры нейронной сети YOLO для обнаружения объектов в реальном времени.

Целью выпускной квалификационной работы является реализация алгоритмов для системы видеонаблюдения, способных автоматически обнаруживать подозрительные объекты и поведение на видеопотоке из аэропортовых камер наблюдения с использованием технологий компьютерного зрения.

Задачи, определяемые поставленной целью:

- Изучение существующих подходов и методов обеспечения безопасности с помощью видеонаблюдения в аэропортах.
- Изучение библиотеки OpenCV и архитектуры YOLO для обнаружения объектов на видеопотоке.
- Подготовка датасета изображений для обучения нейронной сети YOLO на конкретные классы объектов.
- Обучение и настройка модели YOLO на собранном датасете с использованием OpenCV для обработки изображений.
- Реализация методов обнаружения и классификации объектов на видеопотоке из аэропортовых камер наблюдения с использованием обученной модели YOLO и OpenCV.

Объектом исследования является система обеспечения безопасности в аэропортах. Данный объект включает все компоненты и процессы, связанные с обеспечением безопасности в охраняемых зонах аэропортов, включая периметральные оборонительные системы, внутренние видеонаблюдательные системы, процессы проверки на доступ и контрольно-пропускные пункты.

Предметом исследования является реализация системы детектирования и трекинга движущихся объектов с использованием технологий компьютерного зрения в рамках системы обеспечения безопасности аэропортов.

1 Система охранного видеонаблюдения аэропорта

1.1 Проблемы видеонаблюдения в аэропортах

Система видеонаблюдения является неотъемлемой частью обеспечения безопасности в аэропортах. Она позволяет контролировать происходящее на территории, выявлять подозрительную активность и своевременно реагировать на потенциальные угрозы. Однако, несмотря на высокие достижения в области видеонаблюдения, существуют определенные проблемы, которые могут снижать эффективность таких систем.

Аэропорты являются крупными и сложными объектами с множеством зон, включая терминалы, взлетно-посадочные полосы, парковки и служебные помещения. Охват всех данных зон видеокамерами требует значительных ресурсов и продуманного планирования. Установка и обслуживание большого количества камер является трудоемким процессом, требующим значительных финансовых и технических затрат.

Современные системы видеонаблюдения генерируют огромные объемы видеоданных, которые необходимо не только хранить, но и оперативно обрабатывать для выявления потенциальных угроз. Ручной просмотр видеозаписей операторами является неэффективным при таком объеме информации, что обуславливает необходимость использования автоматизированных систем анализа видео. Однако обработка больших объемов данных требует мощных вычислительных ресурсов и сложных алгоритмов, что может вызывать задержки и сбои.

Алгоритмы компьютерного зрения и искусственного интеллекта, используемые для автоматического распознавания объектов и событий, не всегда работают идеально. Ложные срабатывания могут происходить из-за изменения освещения, погодных условий или других факторов, что приводит к ненужным тревогам. В то же время, важные события могут быть пропущены, если алгоритмы недостаточно точны или не адаптированы к специфическим условиям аэропорта.

Системы видеонаблюдения в аэропортах неизбежно затрагивают вопросы конфиденциальности, поскольку камеры фиксируют не только потенциальных нарушителей, но и обычных пассажиров и сотрудников. Соблюдение законодательства о защите персональных данных требует внедрения мер по защите видеопотоков, таких как шифрование и ограничение доступа. Это добавляет дополнительные сложности в управление системой видеонаблюдения.

Эффективная система безопасности аэропорта должна быть интегрирована с другими системами, такими как контроль доступа, пожарная безопасность и системы оповещения. Обеспечение бесшовной интеграции между различными системами требует разработки комплексных решений и стандартов взаимодействия, а также учета возможных конфликтов и несовместимостей между оборудованием и программным обеспечением различных производителей.

Несмотря на высокий уровень автоматизации, человеческий фактор остается важным элементом системы видеонаблюдения. Операторы видеонаблюдения должны быть хорошо обучены и поддерживать высокий уровень концентрации, что может быть затруднительно при длительных сменах и больших объемах информации. Психологические и физические нагрузки на персонал могут приводить к ошибкам в принятии решений и снижению общей эффективности системы.

Проблемы видеонаблюдения в аэропортах требуют комплексного подхода для их решения. Внедрение передовых технологий, таких как искусственный интеллект и машинное обучение, может значительно повысить эффективность и надежность видеонаблюдения. Однако, наряду с технологическими решениями, необходимо уделять внимание организационным аспектам, таким как обучение персонала, разработка процедур и стандартов, а также обеспечение защиты данных и соблюдение прав граждан. Комплексный подход позволит создать эффективную и

надежную систему видеонаблюдения, способную обеспечить высокий уровень безопасности в аэропортах.

1.2 Архитектура системы охранного видеонаблюдения аэропорта

В системе видеонаблюдения охраняемых зон аэропорта интегрируются различные компоненты, обеспечивающие безопасность персонала, пассажиров и имущества. Данная подглава рассматривает основные аспекты структуры данной системы.

Защита охраняемых зон. Камеры наблюдения размещаются на стратегически важных участках для обеспечения непрерывного мониторинга. Данные камеры могут быть фиксированными или поворотными, обеспечивая широкий охват территории и предотвращая несанкционированный доступ.

Обнаружение потенциальных угроз. Система видеонаблюдения обнаруживает подозрительные действия или предметы, позволяя операторам быстро реагировать на возникающие угрозы. Анализ видеопотока помогает выявить аномальные или опасные ситуации. Применение технологий распознавания лиц и автомобильных номеров повышает эффективность обнаружения угроз.

Мгновенные оповещения. Система обеспечивает мгновенное оповещение операторов о возникающих угрозах, автоматически передавая сигналы тревоги. Это позволяет оперативно реагировать на ситуацию и предпринимать необходимые меры для обеспечения безопасности аэропорта.

Структура системы видеонаблюдения в охраняемых зонах аэропорта представляет собой комплексный механизм, обеспечивающий непрерывное наблюдение, обнаружение угроз и оперативное реагирование на них. В современном мире, где угрозы безопасности становятся все более разнообразными и изощренными, системы видеонаблюдения в охраняемых зонах аэропорта играют ключевую роль в обеспечении безопасности персонала, пассажиров и имущества. Пример растановки камер в аэропортах приведен на рис. 1.1.

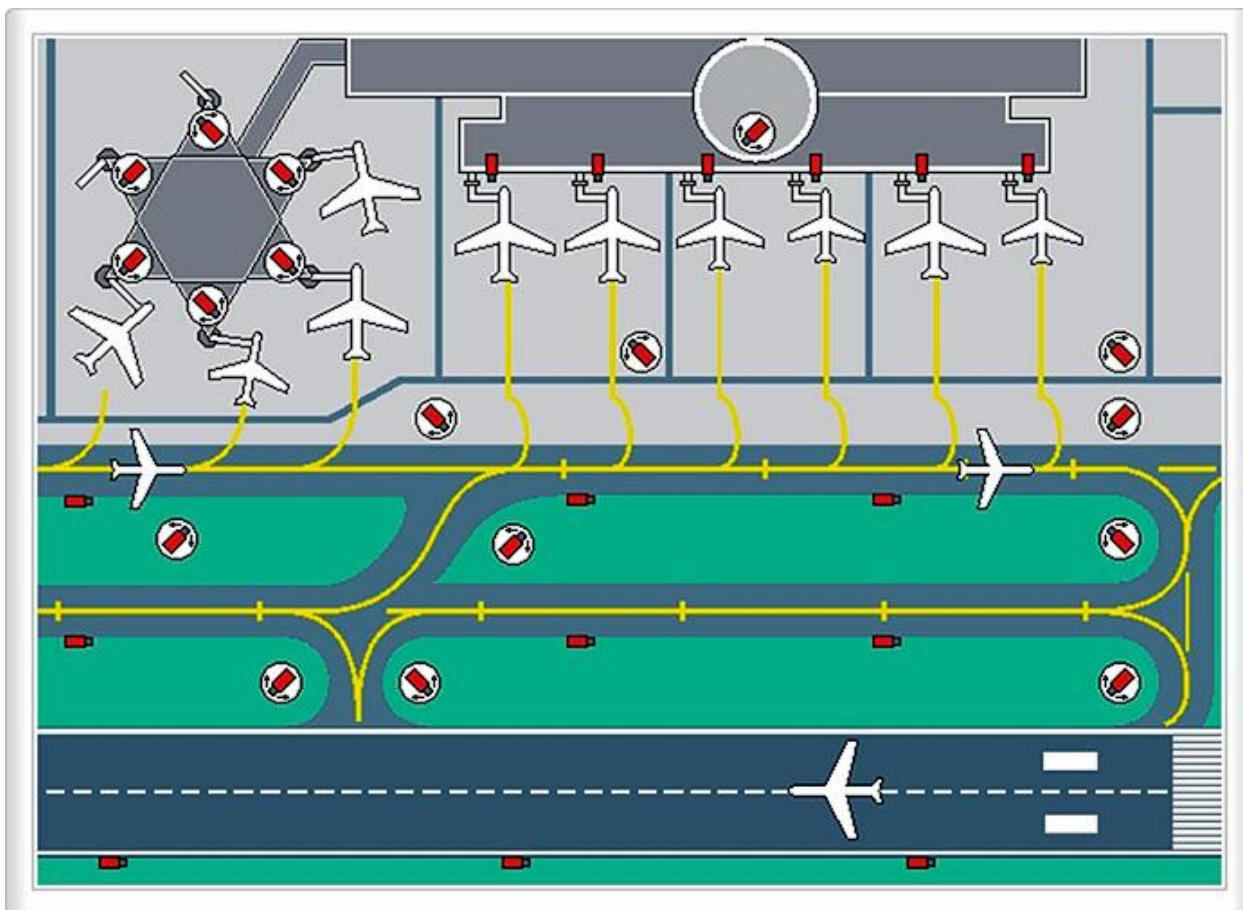


Рисунок 1.1 – Пример установки камер в аэропортах

Однако несмотря на текущий уровень функциональности, существует необходимость постоянного улучшения данной структуры для более эффективного обнаружения и реагирования на потенциальные угрозы. Современные технологии, такие как детектирование, распознавание и трекинг, предоставляют новые возможности для улучшения системы видеонаблюдения в аэропортах.

Возникающие угрозы становятся все более изощренными, требуя более высокой степени автоматизации и точности в обнаружении и идентификации подозрительных действий или объектов. Введение современных технологий, таких как алгоритмы машинного обучения для распознавания лиц и объектов, а также системы трекинга для отслеживания движущихся объектов, позволит значительно улучшить способность системы видеонаблюдения реагировать на потенциальные угрозы. Это позволит операторам системы быстрее и точнее реагировать на инциденты и своевременно предпринимать соответствующие меры по обеспечению безопасности.

1.3 Решаемые задачи обработки видеоданных

В процессе обеспечения безопасности и контроля охраняемых зон аэропорта важными компонентами являются детектирование, распознавание и трекинг объектов. Данные задачи позволяют не только выявлять и идентифицировать движущиеся объекты, но и отслеживать их перемещения, что является ключевым для предотвращения несанкционированного доступа и других нарушений. Рассмотрим данные задачи более подробно.

Детектирование объектов является первым шагом в системе видеонаблюдения. Оно включает в себя процесс обнаружения движущихся объектов в видео потоке. Система должна быть способна эффективно отличать движущиеся объекты от статического фона, что достигается с помощью методов обработки изображений и машинного обучения. Важность детектирования заключается в способности системы выявлять потенциальные угрозы или интересующие объекты в режиме реального времени, что позволяет оперативно реагировать на различные инциденты.

После того как объекты были обнаружены, следующим шагом является их распознавание. Распознавание объектов включает в себя идентификацию и классификацию обнаруженных объектов на основе их характеристик. В контексте аэропортов это может включать распознавание людей, багажа, транспортных средств и других объектов. Точные алгоритмы распознавания позволяют не только идентифицировать тип объекта, но и, при необходимости, проводить дальнейшую проверку, например, лицевую идентификацию пассажиров.

Трекинг объектов завершает процесс, обеспечивая непрерывное отслеживание перемещений обнаруженных и распознанных объектов в пространстве. Трекинг позволяет системе фиксировать траектории движения объектов, что особенно важно для анализа поведения и обнаружения подозрительной активности. Использование алгоритмов трекинга, таких как фильтр Калмана, позволяет предсказывать позиции объектов в последующих кадрах, улучшая стабильность и точность системы видеонаблюдения.

Данные три задачи – детектирование, распознавание и трекинг — являются фундаментальными компонентами эффективной системы видеонаблюдения. Совместное использование данных технологий обеспечивает высокий уровень безопасности и контроля в охраняемых зонах аэропорта, позволяя своевременно реагировать на потенциальные угрозы и предотвращать инциденты.

2 Обнаружение проникновения в охраняемую зону

2.1 Основы детектирования объектов

Детектирование объектов на изображениях и видео является ключевым этапом в обработке компьютерного зрения. Оно заключается в автоматическом обнаружении присутствия объектов определенного класса или категории на изображениях или кадрах видео. Ниже представлен обзор основных методов детектирования объектов:

- Подходы на основе хаар-каскадов (Haar Cascades): Данный метод основан на использовании признаков Хаара, которые представляют собой оконные функции, используемые для выявления особенностей изображений. Алгоритм работает путем сканирования изображения с помощью окон различного размера и формы для поиска соответствующих признаков. Хотя данный метод может быть эффективным для некоторых классов объектов, он обычно требует тщательной настройки параметров для достижения высокой точности детектирования.
- Методы на основе сверточных нейронных сетей (Convolutional Neural Networks, CNN): Современные методы детектирования объектов широко используют глубокие нейронные сети, такие как SSD (Single Shot Multibox Detector), YOLO (You Only Look Once) и Faster R-CNN (Region-based Convolutional Neural Networks). Данные методы представляют собой единую архитектуру, которая способна одновременно предсказывать границы ограничивающих рамок и классы объектов в кадре видео. Они демонстрируют высокую скорость работы и точность детектирования, что делает их предпочтительными для широкого спектра приложений, включая безопасность в аэропортах.
- Методы на основе гистограмм ориентированных градиентов (Histogram of Oriented Gradients, HOG): Данный метод основан на извлечении локальных текстурных и градиентных особенностей из изображения и последующем использовании данных особенностей для обнаружения объектов. Хотя метод HOG был популярен ранее, с развитием глубокого обучения его

применение в детектировании объектов уступило место более современным подходам.

Выбор конкретного метода детектирования объектов зависит от требований конкретной задачи, таких как требуемая скорость работы, точность детектирования, а также доступные ресурсы вычислительной мощности. Для задач безопасности в аэропортах, где необходимо быстро и точно обнаруживать объекты на видео, методы на основе сверточных нейронных сетей часто являются наиболее эффективными. Они позволяют обнаруживать объекты в реальном времени и обеспечивают высокую точность детектирования при минимальных ограничениях по

2.2 Алгоритм детектирования

В контексте дипломной работы, ориентированной на обнаружение движущихся объектов, необходимо стремиться к созданию идеального выходного файла, содержащего видеокadres с правильными обнаружениями. Данный подход предполагает разработку конвейера алгоритма, который позволит достичь поставленной цели (см. рис. 2.1).

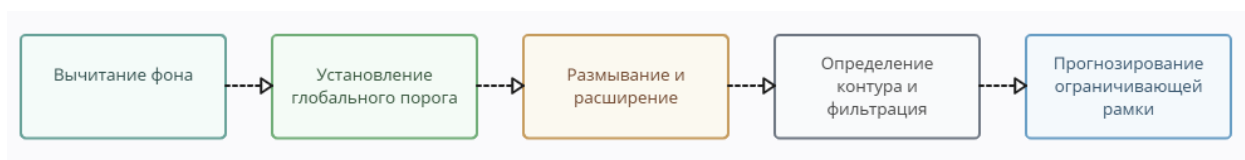


Рисунок 2.1 – Этапы алгоритма детектирования

Алгоритм обнаружения движущихся объектов состоит из нескольких этапов, таких как вычитание фона, установка глобального порога, размытие и расширение, определение контуров, фильтрация и прогнозирование ограничивающих рамок. Рассмотрим каждый из данных шагов более подробно.

2.2.1 Вычитание фона

Первый этап в процессе обнаружения движущихся объектов – вычитание фона. Для этого используется статическая камера, что является распространенным и широко применяемым методом для формирования маски

переднего плана. Данная маска представляет собой двоичное изображение, на котором отображены пиксели, принадлежащие движущимся объектам в сцене.

Фрагмент кода программы с вычитания фона представлен на рис. 2.2.

```
cap = cv2.VideoCapture(vid_path)
backSub = cv2.createBackgroundSubtractorMOG2()
if not cap.isOpened():
    print("Error opening video file")
while cap.isOpened():
    # Захват кадр за кадром
    ret, frame = cap.read()
    if ret:
        # Вычитание фона
        fg_mask = backSub.apply(frame)
```

Рисунок 2.2 – Реализация вычитания фона

Пример записи видеонаблюдения представлен на рис. 2.3.



Рисунок 2.3 – Пример записи видеонаблюдения

Функция вычитания фона вычисляет маску переднего плана путем выполнения операции вычитания между текущим кадром и фоновой моделью, которая содержит статическую часть сцены или все, что можно рассматривать как фон с учетом характеристик сцены. Процесс фоновое моделирование включает в себя два основных этапа: инициализацию фона и его обновление. На первом этапе определяется исходная модель фона, в то время как на втором

этапе эта модель обновляется, чтобы адаптироваться к возможным изменениям в сцене.

2.2.2 Обнаружение контуров

Контуры представляют собой кривые, соединяющие все непрерывные точки с одинаковым цветом или интенсивностью вдоль границы изображения. Они широко используются для анализа формы объектов, их обнаружения и распознавания. OpenCV предоставляет удобные средства для поиска и отображения контуров на изображениях с использованием функций `findContours` и `drawContours`.

Данные функции предлагают два варианта алгоритмов для определения контуров: `CHAIN_APPROX_SIMPLE` и `CHAIN_APPROX_NONE`. Каждый из них имеет свои особенности и может быть выбран в зависимости от конкретных требований приложения.

В результате анализа видеоданных выявляются все движущиеся пиксели между последовательными кадрами. Данные пиксели включают в себя различные объекты, такие как автомобили, тени и маленькие белые точки.

При анализе реальных сценариев различные аспекты окружающей среды могут оказывать влияние на запись видео с камер видеонаблюдения. Данные факторы включают погодные условия, такие как сильный ветер, который может вызывать дрожание камеры или движение объектов, создавая ложные срабатывания датчика движения (см. рис. 2.4).



Рисунок 2.4 – Пример вычитания фона

2.2.3 Рисования контуров

После вычитания фона у нас теперь есть четкая двоичная маска нашей ROI (области интереса). Итак, мы перейдем к обнаружению контура. Приведенный ниже на рис. 2.5 блок кода описывает процесс.

```
# Поиск контура
contours, hierarchy = cv2.findContours(fg_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
# print(contours)
frame_ct = cv2.drawContours(frame, contours, -1, color=(0, 255, 0), thickness=2)
# показать итоговый кадр
cv2.imshow( winname: 'Frame_final', frame_ct)
```

Рисунок 2.5 – Реализация поиска контура

Для этого мы должны передать маску переднего плана (`fg_mask`) в качестве входных данных в `cv2.findContours` функции, чтобы найти все возможные точки контура. Это произойдет для всех обнаруженных объектов в текущем кадре. После этого мы передадим все точки контура в `cv2.drawContours` функцию для рисования контуров для каждого обнаруженного контура.

При рисовании всех контуров становится заметным наличие множества лишних элементов. Данные мелкие, зашумленные контуры могут негативно сказаться на конечных результатах (см. рис. 2.6).

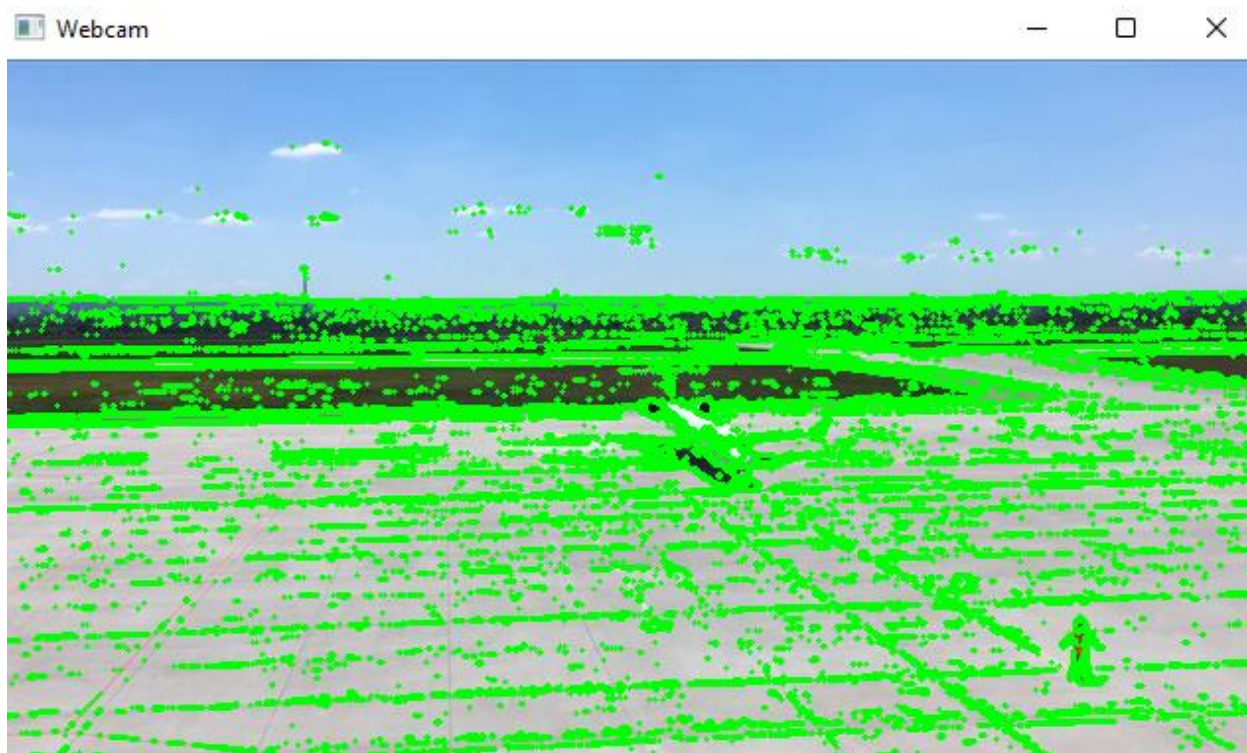


Рисунок 2.6 – Пример наложения маски контуров

2.2.4 Улучшение обнаружения контуров

Появление зашумленных контуров обусловлено движением теней и камеры. Необходимо удалить все помехи, чтобы получить четкий кадр с обнаруженным контуром нашего ROI (автомобиля).

Для решения данной проблемы мы выполняем 3 различных шага:

- 1) Установка порогового значения.
- 2) Применение операций эрозии и расширения.
- 3) Фильтрация контуров.

Для определения порога используется функция `cv2.threshold`, к которой передаются все маски переднего плана в качестве входных данных. Устанавливается пороговое значение на уровне 180, а максимальное значение – на 255. Это приводит к замене всех значений ниже 180 на 0 и удалению всех теней (см. рис. 2.7).

```
# устанавливаем глобальный порог для удаления теней
retval, mask_thresh = cv2.threshold( fg_mask, thresh: 180, maxval: 255, cv2.THRESH_BINARY)
```

Рисунок 2.7 – Установка глобального порога

Результат операции определения порога демонстрируется на полученном изображении (см. рис. 2.8).

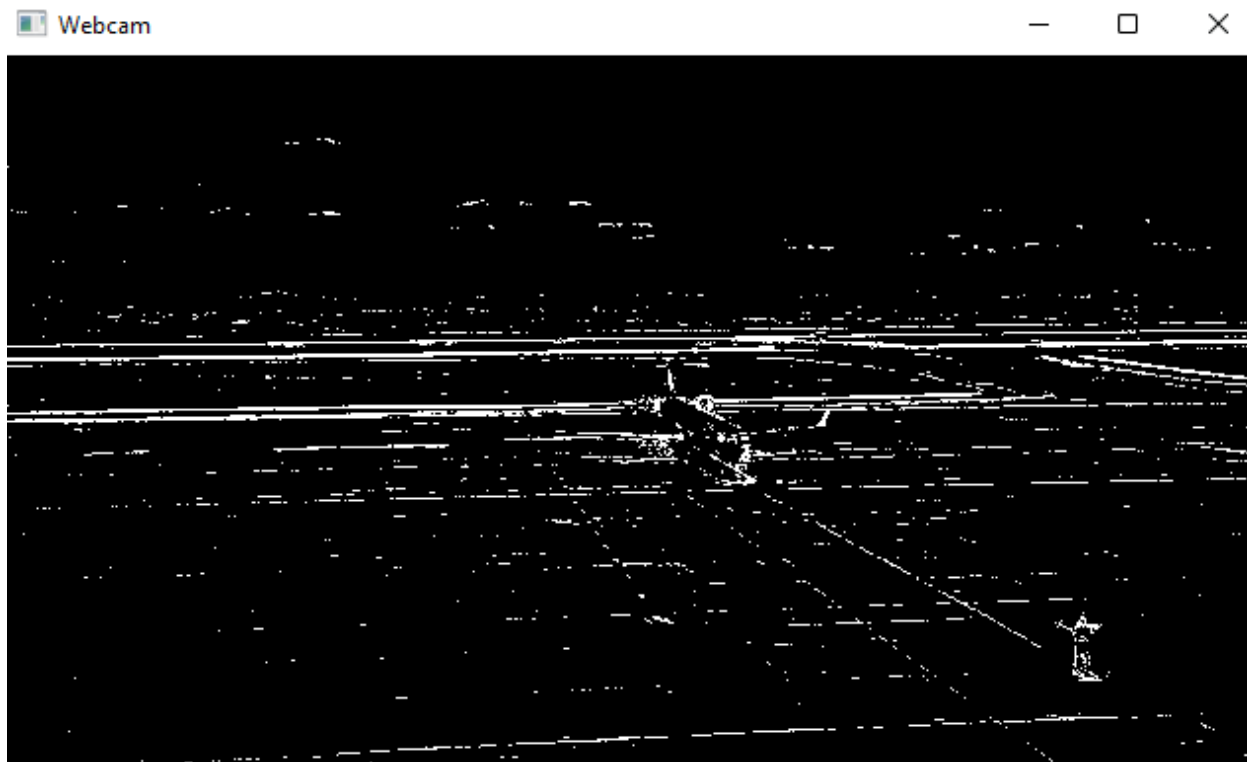


Рисунок 2.8 – Пример улучшения обнаружения контуров

После применения порогового значения у нас формируется маска бинарного изображения, представляющая собой маску переднего плана без теней. Однако в маске остаются маленькие белые точки. Следующим шагом будет удаление контуров вокруг данных маленьких точек.

Для удаления маленьких белых точек применяется операция эрозии, после чего изображение расширяется для получения лучшей и более широкой маски. Для этого используется функция `cv2.morphologyEx` с параметром `cv2.MORPH_OPEN` в качестве типа морфологической операции (см. рис. 2.9).

```
# вычисление ядра
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, ksize=(3, 3))
# Apply erosion
mask_eroded = cv2.morphologyEx(mask_thresh, cv2.MORPH_OPEN, kernel)
```

Рисунок 2.9 – Реализация удаления контуров

В эту операцию передается маска с пороговым значением `mask_thresh`. Результаты обеих операций проверяются. Теперь мы получаем более чистое изображение (см. рис. 2.10). Остальную очистку можно выполнить, отфильтровав контуры небольших участков.



Рисунок 2.10 – Пример операции эрозии

Фильтрация контуров иногда требует применения метода проб и ошибок. Было установлено, что для нашего конкретного случая наилучшим вариантом является использование контура, площадь которого составляет 500 пикселей. В следующем блоке кода производится фильтрация всех контуров, размер которых менее 500 пикселей (см. рис. 2.11). Данный подход позволяет сохранить наиболее заметные объекты (см. рис. 2.12).

```
mask_eroded = cv2.morphologyEx(mask_thresh, cv2.MORPH_OPEN, kernel)
min_contour_area = 500 # Define your minimum area threshold
contours, hierarchy = cv2.findContours(mask_eroded, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
large_contours = [cnt for cnt in contours if cv2.contourArea(cnt) > min_contour_area]
```

Рисунок 2.11 – Реализация фильтрации контуров



Рисунок 2.12 – Пример фильтрации контуров

Следующим этапом в процессе обнаружения движущихся объектов является рисование ограничивающих рамок вокруг оставшихся контуров. Здесь и происходит фактическое обнаружение объектов. Сначала извлекаются координаты прямоугольника с помощью функции `cv2.boundingRect`. Затем для каждого видеокadra рисуются ограничивающие рамки для всех обнаруженных объектов с помощью функции `cv2.rectangle` (см. рис. 2.13).

```
frame_out = frame.copy()
for cnt in large_contours:
    x, y, w, h = cv2.boundingRect(cnt)
    frame_out = cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 3)
```

Рисунок 2.13 – Реализация рисования ограничивающих рамок

Результат операции рисования ограничивающих рамок демонстрируется на полученном изображении (см. рис. 2.14).



Рисунок 2.14 – Пример детектирования движущихся объектов

3 Распознавание движущихся объектов

3.1 Выбор метода распознавания

Для обнаружения и трекинга движущихся объектов в охраняемых зонах аэропортов, в том числе в ангарах с самолетами, эффективным решением является применение алгоритмов компьютерного зрения. Одним из наиболее эффективных и широко применяемых методов обнаружения объектов на видео является YOLO (You Only Look Once).

Преимущества YOLO перед другими методами обнаружения объектов включают в себя:

- 1) Высокая скорость работы: YOLO способен обрабатывать видеопоток в реальном времени благодаря своей способности вычислять расположение и класс объекта в единой операции. Это делает его идеальным выбором для систем мониторинга, где требуется оперативная реакция на изменения ситуации.
- 2) Точность детекции: YOLO демонстрирует высокую точность обнаружения объектов на видео благодаря использованию единой модели для предсказания расположения и класса объекта. Это позволяет избежать проблем, связанных с объединением результатов от различных моделей или этапов обработки.
- 3) Устойчивость к изменениям масштаба: Алгоритм YOLO способен обнаруживать объекты различного размера и формы без необходимости многократного сканирования изображения с разными масштабами.
- 4) Минимальное количество ложных срабатываний: Благодаря своей архитектуре YOLO уменьшает количество ложных срабатываний, что особенно важно для систем безопасности, где каждое ложное срабатывание может привести к неправомерной тревоге или потере ресурсов.
- 5) Простота использования и интеграции: YOLO предоставляет открытый и простой в использовании API, что облегчает его интеграцию с другими

системами безопасности, такими как системы видеонаблюдения в аэропортах.

Выбор YOLO для детектирования и трекинга движущихся объектов в аэропортах обусловлен его высокой производительностью, точностью и устойчивостью к различным условиям эксплуатации. Данные характеристики делают его оптимальным решением для обеспечения безопасности оборудования и жизни людей в охраняемых зонах аэропортов.

3.2 Обзор нейронной сети YOLO

YOLO (You Only Look Once) – одна из самых популярных архитектур для обнаружения объектов в реальном времени. Её особенность заключается в том, что она представляет собой единую нейронную сеть, которая способна предсказывать границы рамок и классы объектов в данных рамках одновременно. Это делает YOLO очень эффективной и быстрой для обработки видеопотоков.

Вот основные шаги по использованию YOLO для обнаружения объектов с использованием OpenCV:

- Загрузка модели YOLO: Сначала необходимо загрузить предварительно обученную модель YOLO. Это можно сделать путем загрузки весов и конфигурационного файла. Кроме того, потребуется файл с метаданными, содержащий названия классов.
- Подготовка изображения: Изображение, на котором вы хотите обнаружить объекты, должно быть предварительно обработано. Это может включать изменение размера, нормализацию и преобразование в формат, который ожидает модель YOLO.
- Прямой проход через сеть: После подготовки изображение подается на вход нейронной сети YOLO для выполнения обнаружения объектов. Это приведет к получению рамок с обнаруженными объектами и их вероятностями принадлежности к классам.
- Отображение результатов: После получения результатов обнаружения объектов и их классов их можно отобразить на изображении с помощью

OpenCV. Это обычно включает отрисовку ограничивающих рамок вокруг объектов и вывод текста с классом и уверенностью модели.

- Отображение и сохранение: Наконец, отображение обработанного изображения с результатами обнаружения объектов и, если необходимо, сохранение его в файл для дальнейшего использования.

Архитектура YOLO имеет сходство с другими методами обнаружения объектов на изображениях в том, что на начальных этапах процесса входное изображение подается на вход сверточной нейронной сети (CNN). В YOLO используется специальная CNN под названием Darknet-53. Далее полученные карты признаков анализируются специальным образом, чтобы определить положение и размеры ограничивающих рамок (bounding boxes) и классы объектов, которым они принадлежат. Разница между двухуровневыми архитектурами и архитектурой YOLO продемонстрирована на рис. 3.1.

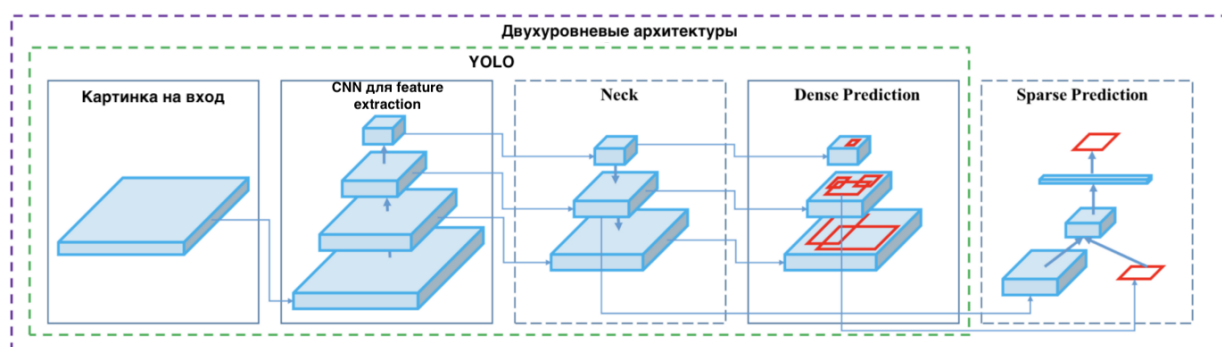


Рисунок 3.1 – Архитектура YOLO

Метод Sparse Prediction повторяет основные принципы двухуровневых алгоритмов, где объекты сначала идентифицируются в отдельных областях, а затем классифицируются данные области.

Блок Neck (или «шея») представляет собой дополнительный компонент, спроектированный для интеграции информации от предыдущих слоев и повышения точности предсказаний. Для более подробного понимания этого аспекта можно обратиться к таким терминам, как «Path Aggregation Network», «Spatial Attention Module» и «Spatial Pyramid Pooling».

Однако ключевым отличием YOLO от других архитектур является блок Dense Prediction. Данный блок является краеугольным камнем YOLO и именно он позволяет алгоритму достичь лидерства в эффективности обнаружения объектов. Философия YOLO (You Only Look Once) заключается в том, что алгоритм проводит анализ изображения всего один раз через нейронную сеть, чтобы выполнить все необходимые шаги по обнаружению объектов.

3.3 Обучение нейронной сети и используемый датасет

Для повышения точности распознавания объектов необходимо обучить нейронную сеть. В качестве моделей для обучения используются изображения и видеозаписи, полученные с камер наблюдения, установленных в охраняемых зонах аэропорта.

Обучение нейронной сети включает следующие этапы:

- 1) Необходимо собрать значительный объем данных, представляющих собой изображения и видеозаписи с различными типами объектов, присутствующих в охраняемых зонах аэропорта. Эти данные должны включать как типичные, так и редкие сценарии, чтобы нейронная сеть могла эффективно распознавать объекты в любых условиях.
- 2) После сбора данных необходимо разметить их, то есть определить границы и классы объектов на каждом изображении или видеокадре. Данная разметка служит эталоном для обучения нейронной сети.
- 3) Размеченные данные необходимо преобразовать в формат, подходящий для обучения нейронной сети YOLO. Это включает нормализацию изображений, изменение их размера и разделение на обучающую и тестовую выборки.
- 4) Обучение модели YOLO проводится с использованием подготовленного датасета. В процессе обучения сеть корректирует свои параметры, чтобы минимизировать ошибку распознавания объектов на изображениях.

- 5) После обучения модель тестируется на отложенной тестовой выборке для оценки её производительности. Данный этап позволяет выявить и устранить возможные ошибки и недочеты в работе модели.

Одним из ключевых факторов, обеспечивающих эффективность YOLOv4, является использование богатого и разнообразного датасета для обучения модели. В контексте YOLOv4 наиболее часто используется датасет COCO (Common Objects in Context).

COCO – это обширный датасет, разработанный для задач детекции объектов, сегментации и описания изображений. Основные характеристики датасета COCO включают:

- Датасет COCO содержит 80 категорий объектов, охватывающих широкий спектр повседневных предметов. К категории объектов относятся такие классы, как люди, животные, транспортные средства, мебель, спортивный инвентарь и многие другие.
- COCO включает более 200 000 изображений, из которых более 80 000 изображений аннотированы с объектами. Всего в датасете насчитывается более 1,5 миллиона аннотаций объектов.
- Каждое изображение в датасете COCO сопровождается аннотациями, включающими координаты ограничивающих рамок (bounding boxes), классы объектов и сегментные маски. Это позволяет использовать датасет для задач как детекции объектов, так и сегментации.
- Важной особенностью COCO является наличие контекстной информации. Объекты размечаются в естественных сценах, что позволяет модели учиться распознавать объекты в различных условиях освещения, углах обзора и с разной степенью перекрытия.

Датасет COCO играет ключевую роль в обеспечении высокой точности и надежности модели YOLOv4. Богатство и разнообразие изображений и аннотаций в COCO позволяют модели эффективно распознавать объекты в различных условиях и контекстах. Использование такого мощного и комплексного датасета способствует достижению выдающихся результатов в

задачах детекции объектов, что делает YOLOv4 одной из ведущих моделей в области компьютерного зрения.

3.4 Подключение обученной модели

Для инициирования вывода на видео, применяется обученная модель. Реализация подключения обученной модели приведен на рис. 3.2.

```
model = YOLO('yolov8n.pt')
net = cv2.dnn.readNet(args.weights, args.config)
blob = cv2.dnn.blobFromImage(image_to_process,
                             1 / 255, size: (608, 608), mean: (0, 0, 0),
                             swapRB=True, crop=False)

net.setInput(blob)
outs = net.forward(out_layers)
```

Рисунок 3.2 – Реализация вывода на видео

Функцией `cv2.dnn.readNet()` можно сгенерировать разные цвета для разных классов, чтобы нарисовать ограничительные рамки.

С видеопотока считывается кадр и подается аргументом в функцию `cv2.dnn.blobFromImage()`, которая подготавливают входное изображение к прогону через глубокую нейронную сеть.

Функция `'net.forward(out_layers)'` выполняет передачу входных данных через нейронную сеть для получения выходных данных.

Параметр `'out_layers'` представляет список имен выходных слоев, которые необходимо получить из модели. После выполнения данной функции получают выходные данные для указанных слоев, которые затем могут быть обработаны для обнаружения объектов на изображении.

Функция `'cv2.dnn.NMSBoxes(boxes, class_scores, 0.0, 0.4)'` в OpenCV применяет алгоритм Non-Maximum Suppression (NMS) к результатам детектирования объектов (см. рис. 3.3).

```
chosen_boxes = cv2.dnn.NMSBoxes(boxes, class_scores, score_threshold: 0.0, nms_threshold: 0.4)
```

Рисунок 3.3 - Реализация chosen_boxes

Данный алгоритм используется для фильтрации дублирующихся детекций и оставления только наиболее значимых объектов.

Параметры:

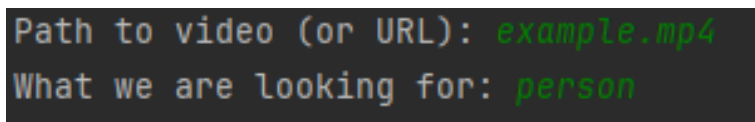
- `'boxes'`: Список ограничивающих рамок (bounding boxes) объектов.
- `'class_scores'`: Список оценок уверенности (confidence scores) для каждого класса объектов.
- `'score_threshold'`: Порог оценки уверенности. Обнаруженные объекты с оценкой уверенности ниже этого порога будут отброшены.
- `'nms_threshold'`: Порог для алгоритма NMS. Данный порог определяет, насколько близко должны быть объекты друг к другу, чтобы считаться дубликатами.

Таким образом, после применения `'cv2.dnn.NMSBoxes()'`, получается отфильтрованный список объектов с их оценками уверенности, где дублирующиеся детекции удалены в соответствии с порогами `'score_threshold'` и `'nms_threshold'`.

Следующими шагами являются прорисовка контуров и вывод количества обнаруженных объектов, за что отвечают пользовательские функции `draw_object_bounding_box` и `draw_object_count` соответственно.

3.4 Результаты

Для тестирования обученной модели в терминал вводится путь к видеофайлу с примером проникновения и имя объекта распознавания.

A screenshot of a terminal window with a dark background. It shows two lines of text in a green monospaced font. The first line is 'Path to video (or URL):' followed by 'example.mp4'. The second line is 'What we are looking for:' followed by 'person'.

```
Path to video (or URL): example.mp4
What we are looking for: person
```

Рисунок 3.4 – Пример ввода пути файла и объект распознавания
Результат работы приложения приведен на рис. 3.5.



Рисунок 3.5 – Пример распознавания объекта

4 Трекинг объектов наблюдения

Для трекинга объектов после их детектирования используется комбинация алгоритмов YOLO и OpenCV. YOLO обеспечивает высокоточную детекцию объектов на изображении или видео, а для отслеживания объектов в последующих кадрах применяются алгоритмы трекинга, такие как алгоритм Кальмана или оптимальная фильтрация данных. Данные алгоритмы позволяют прогнозировать новое положение объекта на основе предыдущих измерений и корректировать его позицию с учетом новых данных.

4.1 Фильтр Калмана

Фильтр Калмана представляет собой мощный инструмент для оценки состояния динамических систем. В контексте задачи отслеживания объектов фильтр Калмана используется для предсказания и обновления положения объектов на основе ограничивающих рамок. Алгоритм позволяет интегрировать шумные измерения и модели движения для достижения более точных предсказаний.

Фильтр Калмана основан на линейных моделях системы и измерений, описываемых следующими уравнениями:

- 1) Модель системы (предсказание состояния):

$$x_k = F \cdot x_{k-1} + B \cdot u_k + w_k, \quad (1)$$

где x_k — вектор состояния в момент времени k , F — матрица перехода состояния, B — матрица управления, u_k — вектор управления, w_k — шум процесса, предполагаемый гауссовским с нулевым средним и ковариацией Q ;

- 2) Модель измерений (обновление состояния):

$$z_k = H \cdot x_k + v_k, \quad (2)$$

где z_k — вектор измерений, H — матрица измерений, v_k — шум измерений, также предполагаемый гауссовским с нулевым средним и ковариацией R .

Алгоритм фильтра Калмана состоит из двух основных этапов: предсказание и обновление.

На этапе предсказания рассчитывается новое состояние объекта на основе предыдущего состояния и модели системы:

- 1) Предсказание состояния:

$$\hat{x}_{k|k-1} = F \cdot \hat{x}_{k-1|k-1} + B \cdot u_k \quad (3)$$

- 2) Предсказание ковариации ошибки:

$$P_{k|k-1} = F \cdot P_{k-1|k-1} \cdot F^T + Q \quad (4)$$

где $\hat{x}_{k|k-1}$ – предсказанное состояние, $P_{k|k-1}$ — предсказанная ковариация ошибки.

На этапе обновления корректируются предсказанные значения на основе новых измерений:

- 1) Вычисление инновации:

$$y_k = z_k - H \cdot \hat{x}_{k|k-1} \quad (5)$$

- 2) Вычисление ковариации инновации:

$$S_k = H \cdot P_{k|k-1} \cdot H^T + R \quad (6)$$

- 3) Вычисление матрицы Калмана:

$$K_k = P_{k|k-1} \cdot H^T \cdot S_k^{-1} \quad (7)$$

- 4) Обновление состояния:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \cdot y_k \quad (8)$$

- 5) Обновление ковариации ошибки:

$$P_{k|k} = (I - K_k \cdot H) \cdot P_{k|k-1} \quad (9)$$

где $\hat{x}_{k|k-1}$ – обновленное состояние, $P_{k|k}$ — обновленная ковариация ошибки, K_k – матрица Калмана, I – единичная матрица.

В контексте отслеживания объектов фильтр Калмана используется для моделирования положения и скорости объекта. Вектор состояния включает координаты центра ограничивающей рамки, размеры рамки и их изменения. Фильтр позволяет предсказывать положение объекта в следующих кадрах, учитывая его движение и корректируя это предсказание на основе новых измерений.

Каждый объект, обнаруженный на видео, отслеживается с помощью отдельного экземпляра класса `KalmanBoxTracker`, который использует фильтр Калмана для предсказания и обновления положения объекта. Алгоритм позволяет эффективно интегрировать данные о движении объекта, снижая влияние шумных измерений и заполняя пропуски между детекциями.

Таким образом, фильтр Калмана является ключевым элементом алгоритма отслеживания, обеспечивая точное и стабильное предсказание положения объектов во времени. Это достигается за счет использования моделей динамики и корректировки на основе новых измерений, что делает алгоритм устойчивым к шуму и пропускам в данных.

4.2 Реализация алгоритма трекинга

В рамках реализации трекера SORT (Simple Online and Realtime Tracking) используется класс `KalmanBoxTracker`. Данный класс реализует фильтр Калмана для отслеживания объектов в видео (см. рис. 5.1).

```
class KalmanBoxTracker(object):
    """ ... """
    count = 0
    # abewley + 1
    def __init__(self, bbox): ...

    # abewley
    def update(self, bbox): ...

    1 usage (1 dynamic) # abewley
    def predict(self): ...

    1 usage (1 dynamic) # abewley
    def get_state(self): ...
```

Рисунок 5.1 – Реализация класса KalmanBoxTracker

Класс `KalmanBoxTracker` содержит следующие основные методы:

- Инициализация: задаются начальные значения состояния и ковариации ошибки, а также создаются необходимые матрицы F , H , Q , R (см. рис. 5.2).

```

def __init__(self, bbox):
    """
    """
    self.kf = KalmanFilter(dim_x=7, dim_z=4)
    self.kf.F = np.array([[1,0,0,0,1,0,0],[0,1,0,0,0,1,0],[0,0,1,0,0,0,1],
                          [0,0,0,1,0,0,0],[0,0,0,0,1,0,0],[0,0,0,0,0,1,0],[0,0,0,0,0,0,1]])
    self.kf.H = np.array([[1,0,0,0,0,0,0],[0,1,0,0,0,0,0],[0,0,1,0,0,0,0],[0,0,0,1,0,0,0]])

    self.kf.R[2:,2:] *= 10.
    self.kf.P[4:,4:] *= 1000.
    self.kf.P *= 10.
    self.kf.Q[-1,-1] *= 0.01
    self.kf.Q[4:,4:] *= 0.01

    self.kf.x[:4] = convert_bbox_to_z(bbox)
    self.time_since_update = 0
    self.id = KalmanBoxTracker.count
    KalmanBoxTracker.count += 1
    self.history = []
    self.hits = 0
    self.hit_streak = 0
    self.age = 0

```

Рисунок 5.2 – Инициализация класса KalmanBoxTracker

- Предсказание: выполняется обновление состояния и ковариации ошибки на основе моделей системы и измерений (см. рис. 5.3).

```

def predict(self):
    """
    """
    if((self.kf.x[6]+self.kf.x[2])<=0):
        self.kf.x[6] *= 0.0
    self.kf.predict()
    self.age += 1
    if(self.time_since_update>0):
        self.hit_streak = 0
    self.time_since_update += 1
    self.history.append(convert_x_to_bbox(self.kf.x))
    return self.history[-1]

```

Рисунок 5.3 – Функция предсказания класса KalmanBoxTracker

- Обновление: корректируются предсказанные значения состояния и ковариации ошибки с учетом новых данных измерений (см. рис. 5.4).

```
def update(self, bbox):
    """ ... """
    self.time_since_update = 0
    self.history = []
    self.hits += 1
    self.hit_streak += 1
    self.kf.update(convert_bbox_to_z(bbox))
```

Рисунок 5.4 – Метод обновления класса KalmanBoxTracker

Класс `Sort` управляет множеством экземпляров `KalmanBoxTracker`, отвечая за их инициализацию, обновление и удаление (см. рис. 5.5).

```
class Sort(object):
    # abewley+1
    def __init__(self, max_age=1, min_hits=3, iou_threshold=0.3):
        """
        Sets key parameters for SORT
        """
        self.max_age = max_age
        self.min_hits = min_hits
        self.iou_threshold = iou_threshold
        self.trackers = []
        self.frame_count = 0

    2 usages # abewley+1 *
    def update(self, dets=np.empty((0, 5))):...
```

Рисунок 5.5 – Реализация класса Sort

Данный класс выполняет следующие функции (см. рис. 5.6):

- Обновление треков: принимает новые детекции, сопоставляет их с существующими треками и обновляет состояние трекеров.
- Создание новых треков: при обнаружении новых объектов инициализируются новые экземпляры `KalmanBoxTracker`.
- Удаление старых треков: удаляются треки, которые не обновлялись в течение определенного количества кадров.

```

def update(self, dets=np.empty((0, 5))):
    self.frame_count += 1
    trks = np.zeros((len(self.trackers), 5))
    to_del = []
    ret = []
    for t, trk in enumerate(trks):...
    trks = np.ma.compress_rows(np.ma.masked_invalid(trks))
    for t in reversed(to_del):
        self.trackers.pop(t)
    matched, unmatched_dets, unmatched_trks = associate_detections_to_trackers(dets, trks, self.iou_threshold)
    for m in matched:
        self.trackers[m[1]].update(dets[m[0], :])
    for i in unmatched_dets:
        trk = KalmanBoxTracker(dets[i,:])
        self.trackers.append(trk)
    i = len(self.trackers)
    for trk in reversed(self.trackers):
        d = trk.get_state()[0]
        if (trk.time_since_update < 1) and (trk.hit_streak >= self.min_hits or self.frame_count <= self.min_hits):
            ret.append(np.concatenate((d, [trk.id+1])).reshape(1,-1))
            i -= 1
        if (trk.time_since_update > self.max_age):
            self.trackers.pop(i)
    if (len(ret)>0):
        return np.concatenate(ret)
    return np.empty((0,5))

```

Рисунок 5.6 – Метод для обновления, создания и удаления треков класса Sort

4.3 Сохранение записей обнаружений в базу данных

Эффективное отслеживание объектов в видео включает не только процесс детекции и трекинга, но и сохранение результатов для последующего анализа. Сохранение данных обнаружений в базу данных позволяет организовать систематическое хранение, доступ и обработку информации. В данной главе рассматривается методология и реализация сохранения координат обнаруженных объектов в базу данных PostgreSQL.

Использование базы данных PostgreSQL обусловлено ее высокой производительностью, надежностью и поддержкой стандарта SQL. Для хранения данных об объектах создается специальная таблица, структура которой определяется следующим образом (см. рис. 5.7):

- obj_id – идентификатор объекта.
- frame – номер кадра, на котором был обнаружен объект.
- x – координата центра объекта по оси X.
- y – координата центра объекта по оси Y.

	id [PK] integer	obj_id integer	frame integer	x integer	y integer
1	1	1	1	353	14
2	2	1	2	353	14
3	3	1	3	353	14
4	4	1	4	353	14
5	5	1	5	354	15
6	6	1	6	355	15
7	7	1	7	356	16
8	8	1	8	357	17
9	9	1	9	357	17
10	10	1	10	358	17
11	11	1	11	359	17
12	12	1	12	360	17
13	13	1	13	361	17
14	14	1	14	361	17
15	15	1	15	362	18

Рисунок 5.7 – Пример записей обнаружений в базе данных

Подключение к базе данных PostgreSQL выполняется с использованием библиотеки `'psycopg2'`. Для установления соединения необходимо указать параметры подключения, такие как имя базы данных, пользователь, пароль, хост и порт. В случае успешного подключения создается объект соединения, который используется для выполнения запросов к базе данных.

Для сохранения координат обнаруженных объектов используется функция, выполняющая вставку данных в таблицу. Функция принимает параметры: идентификатор объекта, номер кадра, координаты x и y. Для выполнения SQL-запроса используется курсор, создаваемый из объекта соединения. После выполнения запроса изменения фиксируются с помощью метода `'commit'`.

В процессе обработки видеопотока производится детекция объектов на каждом кадре. Детекции преобразуются в формат, подходящий для трекера. Трекер обновляет состояние объектов, и для каждого объекта вычисляются координаты центра. Данные координаты сохраняются в базу данных с использованием функции вставки.

Сохранение данных об обнаруженных объектах в базе данных обеспечивает возможность последующего анализа и использования информации. Реализованный метод позволяет эффективно организовать процесс сохранения данных в реальном времени, поддерживая интеграцию с системами обработки видеопотоков и трекинга объектов. База данных PostgreSQL, используемая для хранения, предоставляет надежные механизмы для управления данными и выполнения запросов.

4.4 Уведомление операторам об обнаружении

Модуль сигнализации реализует функцию, которая отображает окно уведомления на экране оператора при обнаружении целевого объекта. Для этого используется библиотека OpenCV. Реализация уведомления об обнаружении движущегося объекта приведена на рис. 5.8.

```
def show_alert():  
    root = tk.Tk()  
    root.withdraw() # Скрыть главное окно  
    messagebox.showinfo( title: "Обнаружение объекта", message: "Человек обнаружен!")  
    root.destroy()
```

Рисунок 5.8 – Реализация уведомления об обнаружении

Реализация системы сигнализации и уведомления операторов позволяет оперативно реагировать на появление целевых объектов в зоне наблюдения. Введение дополнительного модуля, ответственного за генерацию и отображение уведомлений, значительно повышает эффективность мониторинга и обеспечивает своевременное информирование операторов о важных событиях. Данная система может быть интегрирована в различные приложения видеонаблюдения, обеспечивая надежную и непрерывную работу в режиме реального времени. Пример для уведомления операторам об обнаружении человека в кадре продемонстрирован на рис. 5.9.

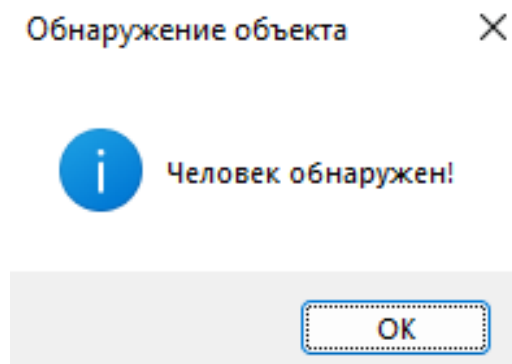


Рисунок 5.9 – Пример уведомления

Уведомление показывается для одного объекта только один раз благодаря использованию множества `'detected_objects'`. Множество служит для хранения идентификаторов обнаруженных объектов. Логика работы кода следующая:

- В начале основного цикла, где производится обработка каждого кадра видео, инициализируется пустое множество `'detected_objects'`. Оно будет использоваться для хранения уникальных идентификаторов обнаруженных объектов.
- Для каждого отслеживаемого объекта проверяется, находится ли его идентификатор `'obj_id'` в множестве `'detected_objects'`.
- Если `'obj_id'` не найден в множестве, это означает, что объект обнаружен впервые. В этом случае идентификатор объекта добавляется в множество `'detected_objects'`, и вызывается функция `'show_alert'`, которая показывает уведомление об обнаружении объекта.

Это гарантирует, что уведомление показывается только один раз для каждого уникального объекта, поскольку после первого показа его идентификатор добавляется в множество `'detected_objects'`, и последующие проверки для этого объекта будут всегда возвращать `'True'`, предотвращая повторное отображение уведомления.

4.5 Результаты

Сочетание быстрой и точной детекции объектов с последующим трекингом позволяет эффективно отслеживать объекты на

видеоизображениях. Графический вывод результатов и сохранение записей обнаружений в базу данных делают процесс анализа и визуализации полученных данных более удобным и информативным.

Результат применения методов трекинга для объекта, стоящего на одном месте, приведен на рис. 5.10.



Рисунок 5.10 – Пример трекинга 1

Результаты работы метода трекинга для другого видео, где объект перемещается в кадре, приведены на рис. 5.11 и рис. 5.12.

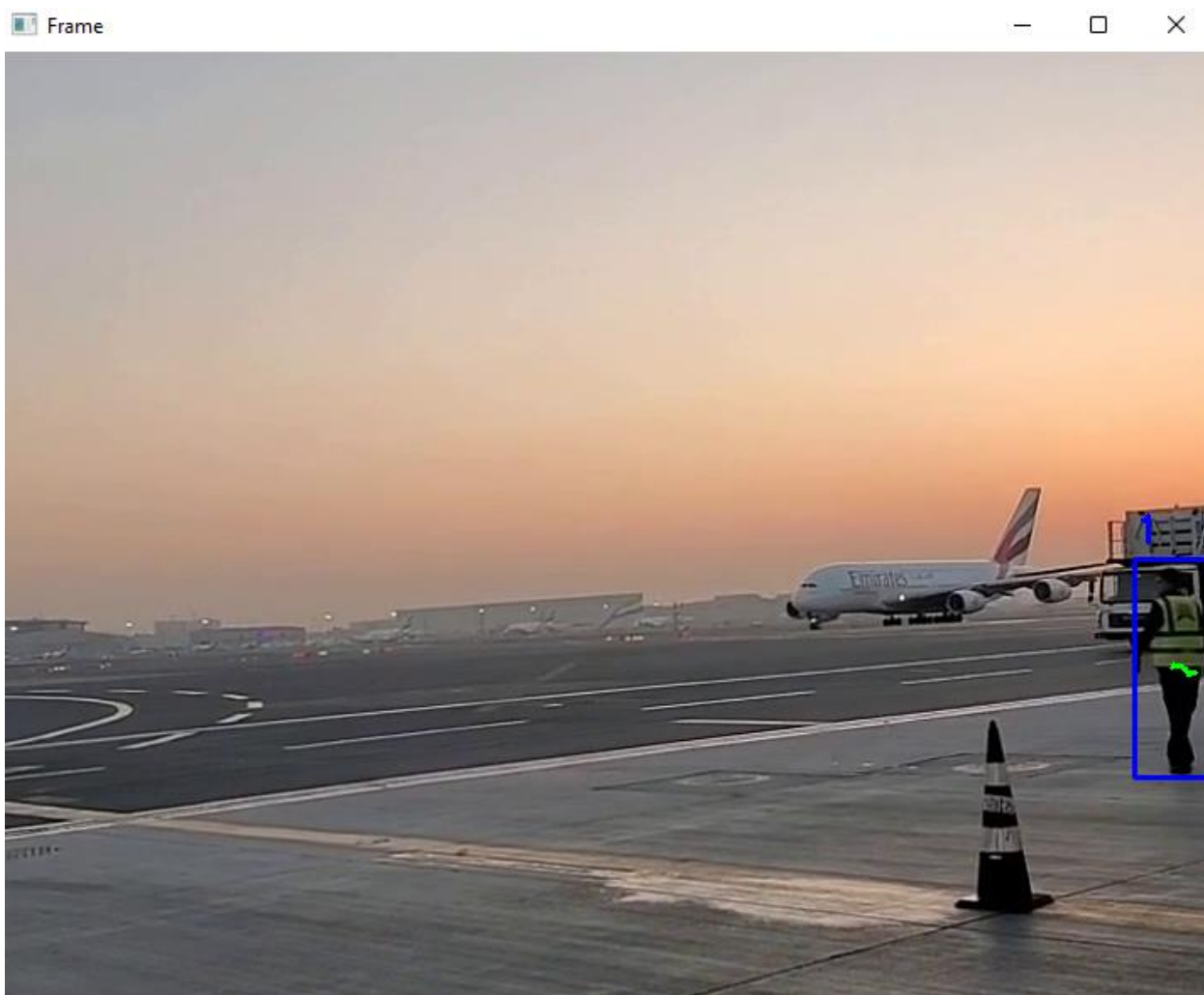


Рисунок 5.11 – Пример с трекингом объекта 2

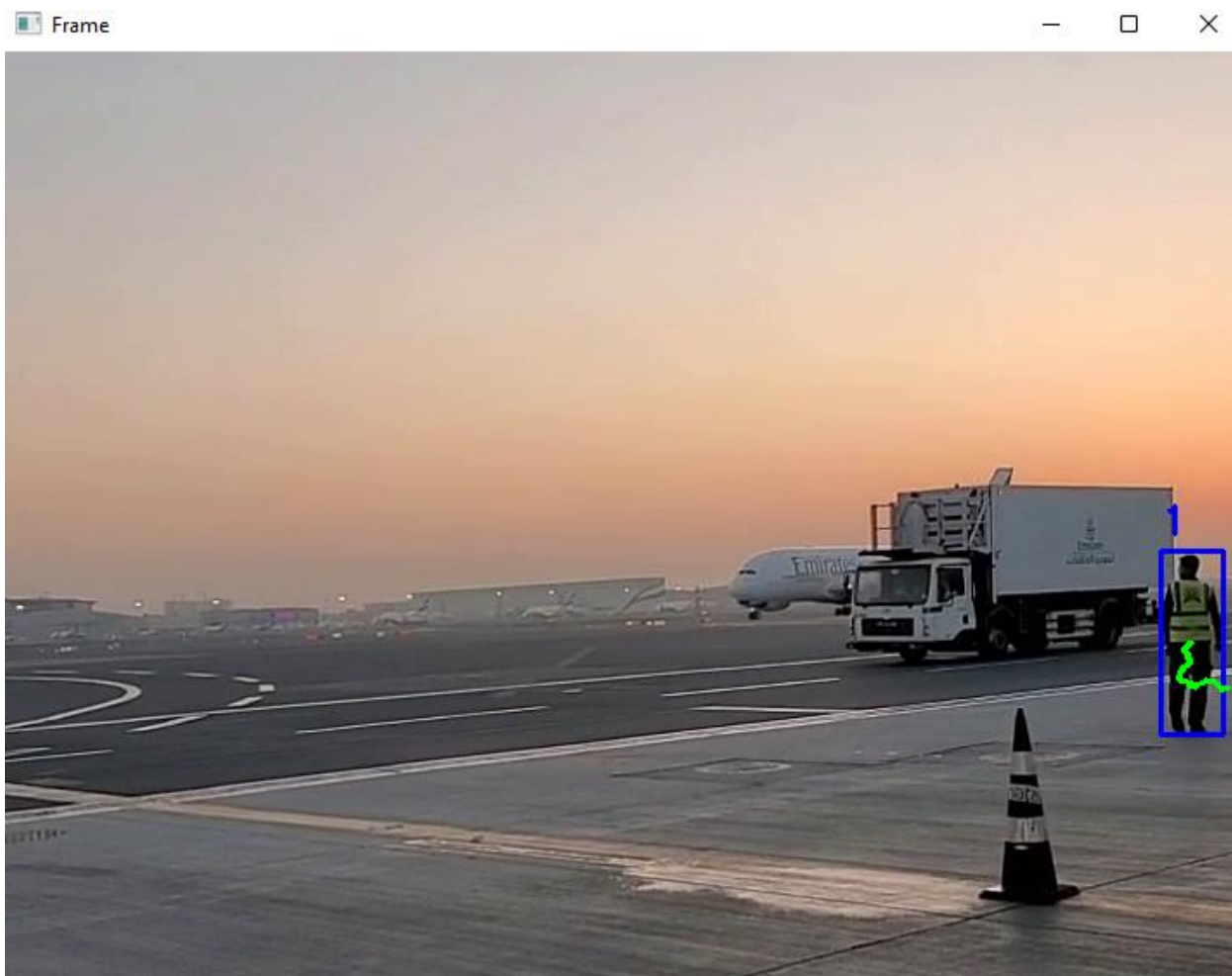


Рисунок 5.12 – Пример с трекингом объекта 3

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы рассмотрены вопросы построения систем видеонаблюдения для охраняемых зон аэропорта с применением современных технологий компьютерного зрения и машинного обучения. Основное внимание уделено алгоритмам детектирования, распознавания и трекинг движущихся объектов, их программной реализации на языке python с применением библиотеки OpenCV.

Основные полученные в работе результаты:

- решена задача детектирования движущихся объектов в зоне видеонаблюдения, основу которой составляют методы вычитания фона и как выделение контуров;
- подготовка датасета, включающий тысячи размеченных изображений, связанных с рассматриваемой предметной областью – охраной территории аэропорта. Созданный датасет стал основой для обучения модели YOLO.
- решена задача распознавания движущихся объектов в зоне видеонаблюдения аэропорта с применением нейронной сети YOLO позволяющей распознавать объекты в реальном времени.
- решена задача трекинга объектов наблюдения решена с использованием фильтра Калмана.

Работоспособность и эффективность разработанных алгоритмов и программ подтверждена вычислительными экспериментами.

Практическая значимость выполненной работы заключается в применении современных технологий к системе видеонаблюдения охраняемых зон, которая способна значительно повысить уровень безопасности в аэропортах. Автоматизация процессов детектирования, распознавания и трекинга объектов позволяет операторам быстро реагировать на потенциальные угрозы, снижая риск инцидентов и повышая общую эффективность системы безопасности.

СПИСОК ЛИТЕРАТУРЫ

1. Садыхов Р.Х., Кучук С.А. Системы видеонаблюдения. Состояние, проблемы и технические средства обработки изображений// Информатика. 2013. № 3. – С. 35.
2. OpenCV Documentation. OpenCV. URL: <https://docs.opencv.org/> (дата обращения: 25.01.2024).
3. Солем Я.Э. Программирование компьютерного зрения на языке Python / пер. с англ. Слинкин А.А. – М.: ДМК Пресс, 2016. – 312 с.
4. Хаус Дж. Изучение компьютерного зрения с OpenCV 4 с Python. – М.: Питер, 2019. – 400 с.
5. OpenCV Python Tutorials. OpenCV. URL: https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_tutorials.html (дата обращения: 02.02.2024).
6. Фернандес В.А. Овладение OpenCV 4 с Python. – М.: Питер, 2019. – 450 с.
7. Джоши П. OpenCV на практике. – М.: ДМК Пресс, 2017. – 320 с.
8. Обнаружение движущихся объектов с помощью OpenCV с использованием обнаружения контуров и вычитания фона // Habr. URL: <https://habr.com/ru/articles/786436/> (дата обращения: 15.02.2024).
9. Ханссон Н. Компьютерное зрение в реальном времени с OpenCV. - М.: Вильямс, 2018. – 360 с.
10. Редмонд Дж., Дивала С. YOLO: You Only Look Once. URL: <https://arxiv.org/abs/1506.02640> (дата обращения: 17.02.2024).
11. Бошковски А., Ванг Ч. YOLOv4: Оптимальная скорость и точность обнаружения объектов. URL: <https://arxiv.org/abs/2004.10934> (дата обращения: 01.03.2024).
12. YOLOv5 Documentation. Ultralytics. URL: <https://docs.ultralytics.com/> (дата обращения: 17.02.2024).
13. YOLOv3 и YOLOv4: реализация алгоритмов обнаружения объектов. Towards Data Science. URL: <https://towardsdatascience.com/yolo-v3-object-detection-53fb7d3bfe6b> (дата обращения: 05.03.2024).

- 14.Власов С.О., Гладышев А.И., Богуславский А.А., Соколов С.М. Решение задачи обнаружения объекта с помощью нейросетевых технологий // Препринты ИПМ им. М.В.Келдыша 2023. № 16. – С. 1-27.
- 15.Роузброк А. Глубокое обучение для компьютерного зрения с Python / Адриан Роузброк. – М.: Питер, 2020. – 500 с.
- 16.Уэлч Г., Бишоп Г. Введение в фильтр Калмана. URL: https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf (дата обращения: 05.03.2024).
- 17.Лаббе Р. Фильтры Калмана и Байесовские фильтры на Python. URL: <https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python> (дата обращения: 15.03.2024).
- 18.Трун С., Бургард В., Фокс Д. Вероятностная робототехника. – М.: Бином, 2005. – 650 с.
- 19.Гревал М. Теория и практика фильтра Калмана с использованием MATLAB / Мохиндер С. Гревал. – М.: Вильямс, 2014. – 480 с.
- 20.Саймон Д. Понимание основ фильтра Калмана через простое и интуитивное выведение. URL: https://academic.csuohio.edu/simond/understanding_the_kalman_filter.pdf (дата обращения: 15.03.2024).
- 21.Хайкин С. Фильтры Калмана и нейронные сети / Саймон Хайкин. - М.: Вильямс, 2002. - 520 с.
- 22.Расширенный фильтр Калмана (EKF) учебник. UNC. URL: <http://www.cs.unc.edu/~welch/kalman/> (дата обращения: 17.03.2024).

QR-код на страницу проекта в GitHub



Рисунок П.1 - QR-код на страницу проекта в GitHub